Ziyan_A8 Code

Step1:

Networking Configuration

main.tf

```
provider "aws" {
  region = "us-east-1"
}

# 1. Create VPC
resource "aws_vpc" "webapp_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "webapp-vpc"
  }
}

# 2. Create Public Subnets
resource "aws_subnet" "public_1" {
  vpc_id            = aws_vpc.webapp_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet-1"
  }
}

resource "aws_subnet" "public_2" {
  vpc_id            = aws_vpc.webapp_vpc.id
  cidr_block        = "10.0.2.0/24"
  availability_zone = "us-east-1b"
  map_public_ip_on_launch = true
```

```
  tags = {
    Name = "public-subnet-2"
  }
}

resource "aws_subnet" "public_3" {
  vpc_id            = aws_vpc.webapp_vpc.id
  cidr_block        = "10.0.3.0/24"
  availability_zone = "us-east-1c"
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet-3"
  }
}

# 3. Create Private Subnets
resource "aws_subnet" "private_1" {
  vpc_id            = aws_vpc.webapp_vpc.id
  cidr_block        = "10.0.4.0/24"
  availability_zone = "us-east-1a"
  tags = {
    Name = "private-subnet-1"
  }
}

resource "aws_subnet" "private_2" {
  vpc_id            = aws_vpc.webapp_vpc.id
  cidr_block        = "10.0.5.0/24"
  availability_zone = "us-east-1b"
  tags = {
    Name = "private-subnet-2"
  }
}

# 4. Create Internet Gateway
resource "aws_internet_gateway" "igw" {
```

```
  vpc_id = aws_vpc.webapp_vpc.id
  tags = {
    Name = "webapp-igw"
  }
}

# 5. Create Public Route Table
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.webapp_vpc.id
  tags = {
    Name = "public-route-table"
  }
}

# Add route to IGW
resource "aws_route" "public_internet_access" {
  route_table_id         = aws_route_table.public_rt.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.igw.id
}

# Associate Public Subnets with Public Route Table
resource "aws_route_table_association" "public_rt_assoc_1" {
  subnet_id      = aws_subnet.public_1.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "public_rt_assoc_2" {
  subnet_id      = aws_subnet.public_2.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "public_rt_assoc_3" {
  subnet_id      = aws_subnet.public_3.id
  route_table_id = aws_route_table.public_rt.id
}
```

```
# 6. Create Private Route Table (no internet route)
resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.webapp_vpc.id
  tags = {
    Name = "private-route-table"
  }
}

# Associate Private Subnets with Private Route Table
resource "aws_route_table_association" "private_rt_assoc_1" {
  subnet_id      = aws_subnet.private_1.id
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_route_table_association" "private_rt_assoc_2" {
  subnet_id      = aws_subnet.private_2.id
  route_table_id = aws_route_table.private_rt.id
}
```

step2

Security Groups

main.tf

```
#Security Group for ALB
resource "aws_security_group" "alb_sg" {
  name        = "alb-sg"
  description = "Allow HTTP from anywhere"
  vpc_id      = aws_vpc.webapp_vpc.id

  ingress {
```

```
    description = "Allow HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow all outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "ALB Security Group"
  }
}

#Security Group for EC2
resource "aws_security_group" "ec2_sg" {
  name        = "ec2-sg"
  description = "Allow HTTP from ALB"
  vpc_id      = aws_vpc.webapp_vpc.id

  ingress {
    description     = "Allow HTTP from ALB SG"
    from_port       = 8080
    to_port         = 8080
    protocol        = "tcp"
    security_groups = [aws_security_group.alb_sg.id]
  }

  egress {
    description = "Allow all outbound"
```

```
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "EC2 Security Group"
  }
}

#Security Group for RDS
resource "aws_security_group" "rds_sg" {
  name        = "rds-sg"
  description = "Allow MySQL from EC2 SG"
  vpc_id      = aws_vpc.webapp_vpc.id

  ingress {
    description     = "Allow MySQL from EC2 SG"
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.ec2_sg.id]
  }

  egress {
    description = "Allow all outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "RDS Security Group"
```

```
  }
}
```

Step3

Provision RDS Instance for the Web Application

main.tf

```
# Create a DB subnet group using the two private subnets
resource "aws_db_subnet_group" "rds_subnet_group" {
  name       = "webapp-db-subnet-group"
  subnet_ids = [
    aws_subnet.private_1.id,
    aws_subnet.private_2.id
  ]
  tags = {
    Name = "webapp-db-subnet-group"
  }
}

# RDS Parameter Group
resource "aws_db_parameter_group" "webapp_pg" {
  name        = "webapp-pg"
  family      = "postgres16"
  description = "Custom parameter group for webapp"

  parameter {
    name  = "rds.force_ssl"
    value = "0"
  }
}

# RDS Instance
resource "aws_db_instance" "webapp_db" {
  identifier             = "webapp-db"
```

```
  allocated_storage     = 20
  storage_type          = "gp2"
  engine                = "postgres"
  engine_version        = "16"
  instance_class        = "db.t3.micro"
  db_name               = "webapp_db"
  username              = "adminuser"
  password              = "password123!"
  db_subnet_group_name   = aws_db_subnet_group.rds_subnet_group.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  publicly_accessible    = false
  skip_final_snapshot    = true
  parameter_group_name   = aws_db_parameter_group.webapp_pg.name
  multi_az              = false

  tags = {
    Name = "webapp-rds"
  }
}
```

step4

EC2 Instances Running the Web Application

main.tf

```
# Create Launch Template
resource "aws_launch_template" "webapp_lt" {
  name_prefix   = "webapp-lt-"
  image_id      = "ami-03a6c16a66bbe0a7a"
  instance_type = "t3.micro"

  vpc_security_group_ids = [aws_security_group.ec2_sg.id]
```

```
  user_data = base64encode(<<EOF
#!/bin/bash
mkdir -p /usr/bin/csye6225

cat <<EOT > /usr/bin/csye6225/.env
DB_HOST=${aws_db_instance.webapp_db.address}
DB_PORT=3306
DB_USERNAME=${aws_db_instance.webapp_db.username}
DB_PASSWORD=${aws_db_instance.webapp_db.password}
DB_NAME=${aws_db_instance.webapp_db.db_name}
EOT

chmod 600 /usr/bin/csye6225/.env
chown csye6225:csye6225 /usr/bin/csye6225/.env

sudo systemctl start webapp.service
EOF
  )

  tag_specifications {
    resource_type = "instance"
    tags = {
      Name = "webapp-instance"
    }
  }
}
```

step5

Application Load Balancer (ALB) Configuration

main.tf

```
# Target Group
resource "aws_lb_target_group" "webapp_tg" {
  name     = "webapp-target-group"
  port     = 8080
  protocol = "HTTP"
  vpc_id   = aws_vpc.webapp_vpc.id
  target_type = "instance"

  health_check {
    protocol = "HTTP"
    path     = "/healthz"
    interval = 30
    timeout  = 5
    healthy_threshold   = 2
    unhealthy_threshold = 2
  }

  tags = {
    Name = "webapp-target-group"
  }
}

# Application Load Balancer (ALB)
resource "aws_lb" "webapp_alb" {
  name               = "webapp-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb_sg.id]
  subnets            = [
    aws_subnet.public_1.id,
    aws_subnet.public_2.id,
    aws_subnet.public_3.id
  ]

  tags = {
```

```
    Name = "webapp-alb"
  }
}

# Listener: ALB → Target Group
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.webapp_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.webapp_tg.arn
  }
}
```

step6

main.tf

```
#create ASG
resource "aws_autoscaling_group" "webapp_asg" {
  name                 = "webapp-asg"
  min_size             = 1
  max_size             = 3
  desired_capacity     = 1
  vpc_zone_identifier  = [
    aws_subnet.public_subnet_1.id,
    aws_subnet.public_subnet_2.id,
    aws_subnet.public_subnet_3.id
  ]
  health_check_type    = "ELB"
  health_check_grace_period = 60
  target_group_arns    = [aws_lb_target_group.webapp_tg.arn]
```

```
  launch_template {
    id      = aws_launch_template.webapp_lt.id
    version = "$Latest"
  }

  tag {
    key                 = "Name"
    value               = "WebApp Instance"
    propagate_at_launch = true
  }

  lifecycle {
    create_before_destroy = true
  }
}
```

```
# Scale Up Policy
resource "aws_autoscaling_policy" "scale_up" {
  name                   = "scale-up-policy"
  scaling_adjustment     = 1
  adjustment_type        = "ChangeInCapacity"
  cooldown               = 300
  autoscaling_group_name = aws_autoscaling_group.webapp_asg.name
}

resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name          = "high-cpu-alarm"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 120
  statistic           = "Average"
  threshold           = 70
```

```
  alarm_description   = "Scale up if CPU > 70%"
  dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.webapp_asg.name
  }
  alarm_actions = [aws_autoscaling_policy.scale_up.arn]
}

# Scale Down Policy
resource "aws_autoscaling_policy" "scale_down" {
  name                   = "scale-down-policy"
  scaling_adjustment     = -1
  adjustment_type        = "ChangeInCapacity"
  cooldown               = 300
  autoscaling_group_name = aws_autoscaling_group.webapp_asg.name
}

resource "aws_cloudwatch_metric_alarm" "cpu_low" {
  alarm_name          = "low-cpu-alarm"
  comparison_operator = "LessThanThreshold"
  evaluation_periods  = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 120
  statistic           = "Average"
  threshold           = 30
  alarm_description   = "Scale down if CPU < 30%"
  dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.webapp_asg.name
  }
  alarm_actions = [aws_autoscaling_policy.scale_down.arn]
}
```